

RAG

Satya Pattnaik

Why a RAG?

train - incontext learning } X

→ avoid LLM hallucinations

→ ground the model with factual



RAG
+
finetuning LLM

Show me a way to train T5?

Pre-deployment

$Emb(q)$

Key	Value
$Emb(Doc1)$	Doc1
$Emb(Doc2)$	Doc2
$\vdots$	$\vdots$

- Offline
- Online

Offline

Huggingface docs (2000)

Additional documents

1

Encode

Embedding Model

2

Index



Vector database

4

Similarity search

Cosine Similarity

Similar documents

Doc2



User Query + Doc2

Prompt

6



LLM

7

Response

Response

Grounded response

(real time) Online

Query

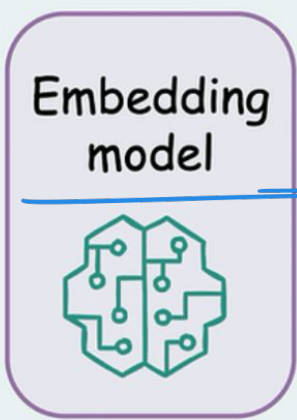


Tell me how to train TS?

3

Encode

Embedding Model

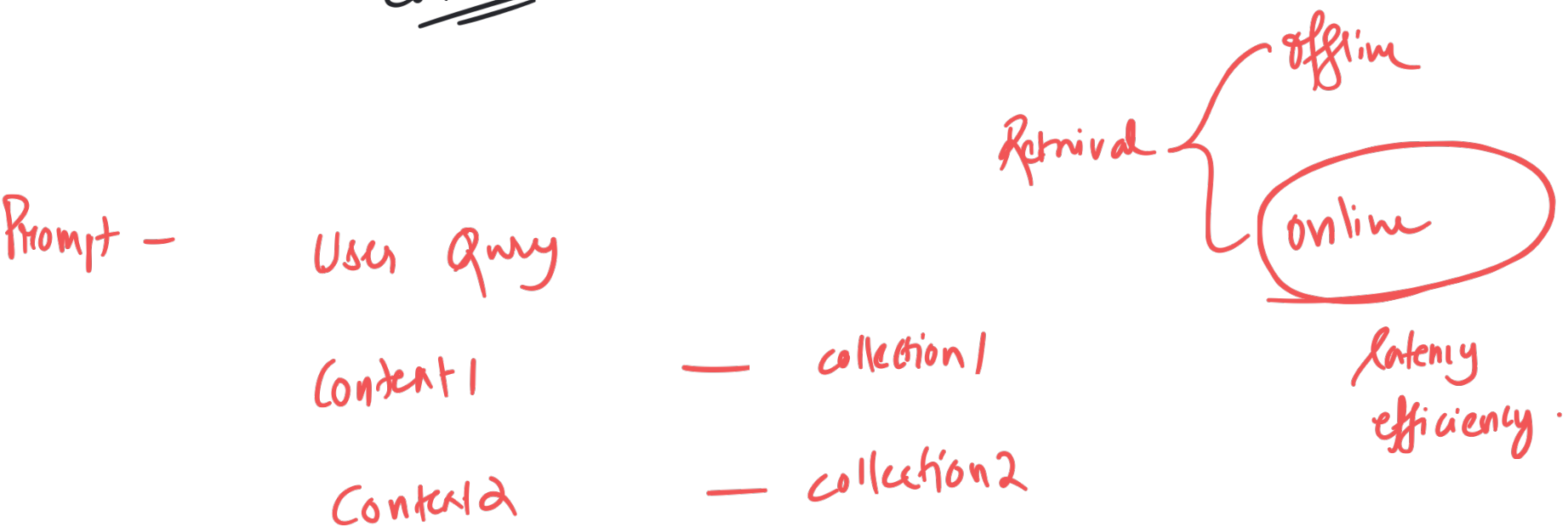
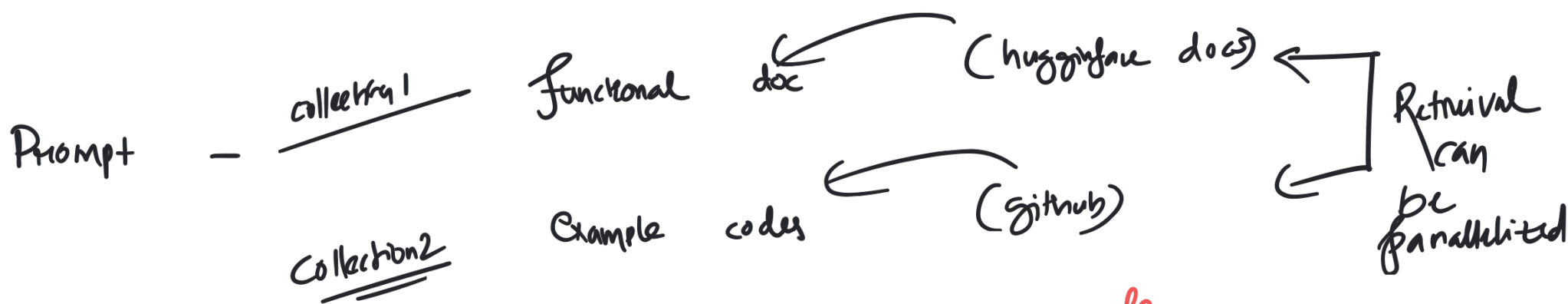


User query

Context

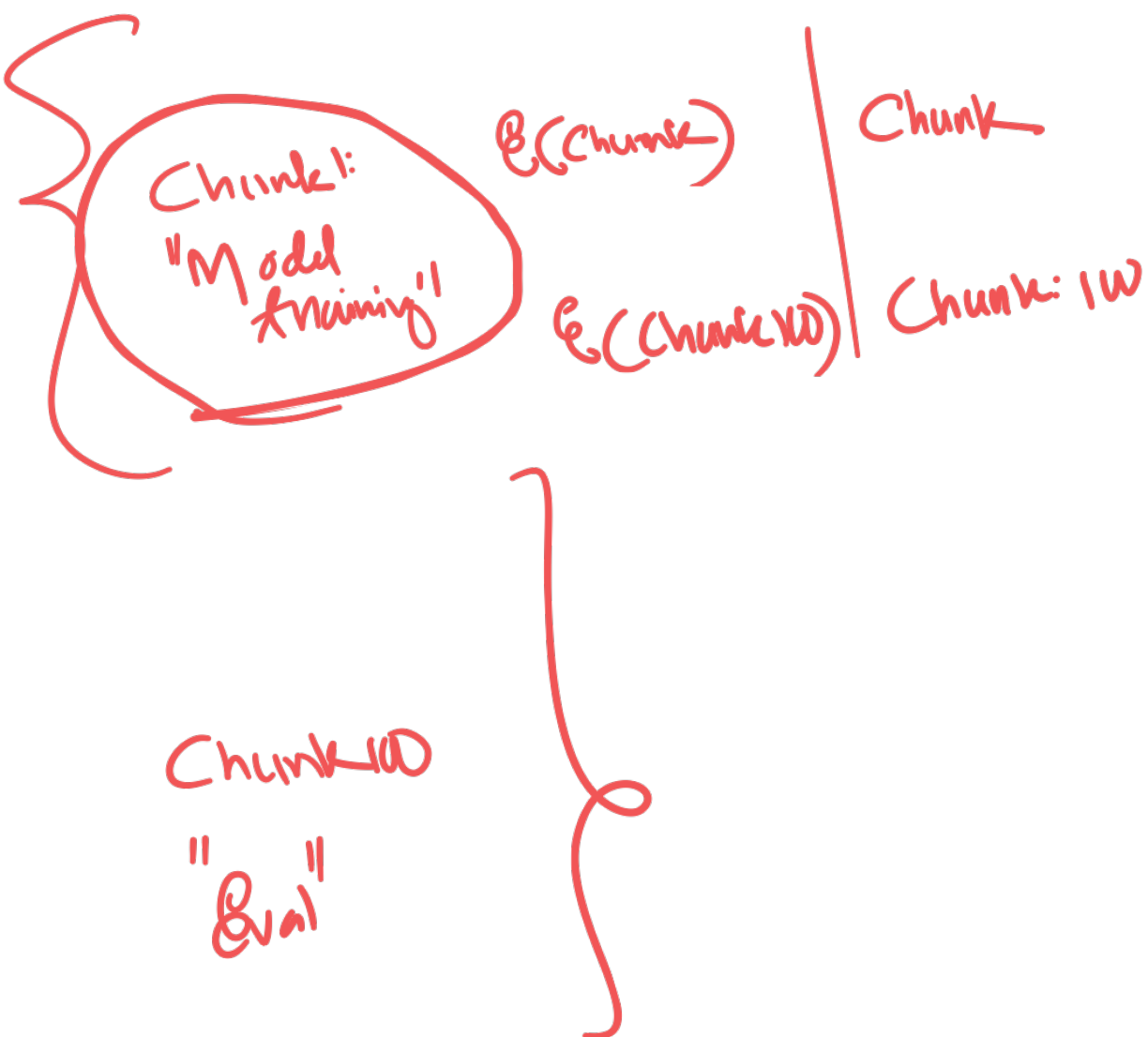
Prompt

RAG



Metadata - Model train

+ Vector/Semantic Search

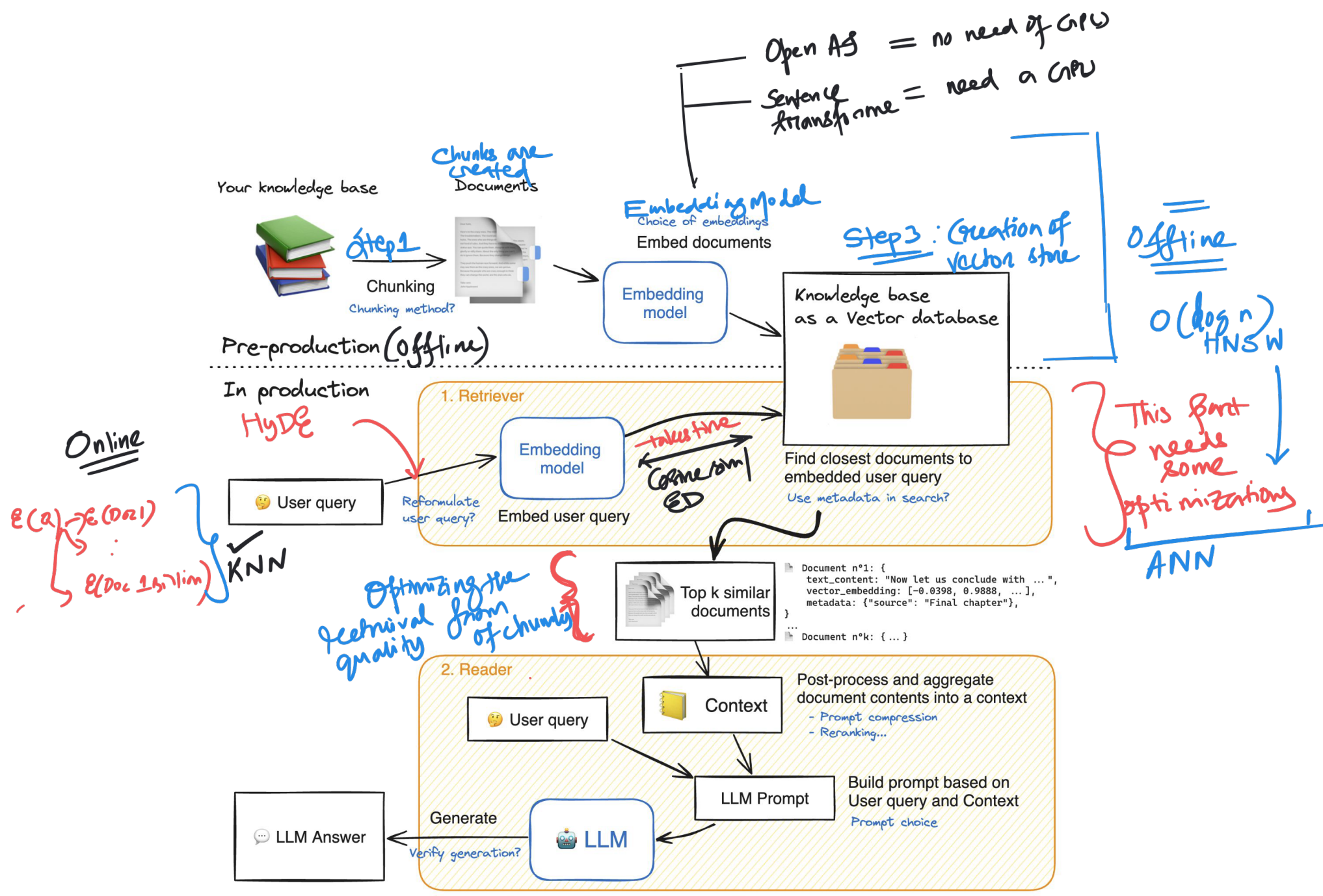


$$e(a) \begin{bmatrix} 1 \times 348 \end{bmatrix} \cdot \begin{bmatrix} \end{bmatrix}$$

1 shot operation

$$\begin{bmatrix} \end{bmatrix}_{33\ 000 \times 348} \begin{bmatrix} 33\ 000 \\ 0.2 \\ 0.9 \\ 0.4 \\ \vdots \\ \vdots \\ \vdots \end{bmatrix}$$

$$\{ \underline{\underline{9.32}} \}$$



Chunking

process of splitting large docs — smaller/meaningful pieces
(generally before embedding it)

- Why?
- LLMs have limited context windows
 - large docs overwhelm/confuse model
 - important info might get diluted

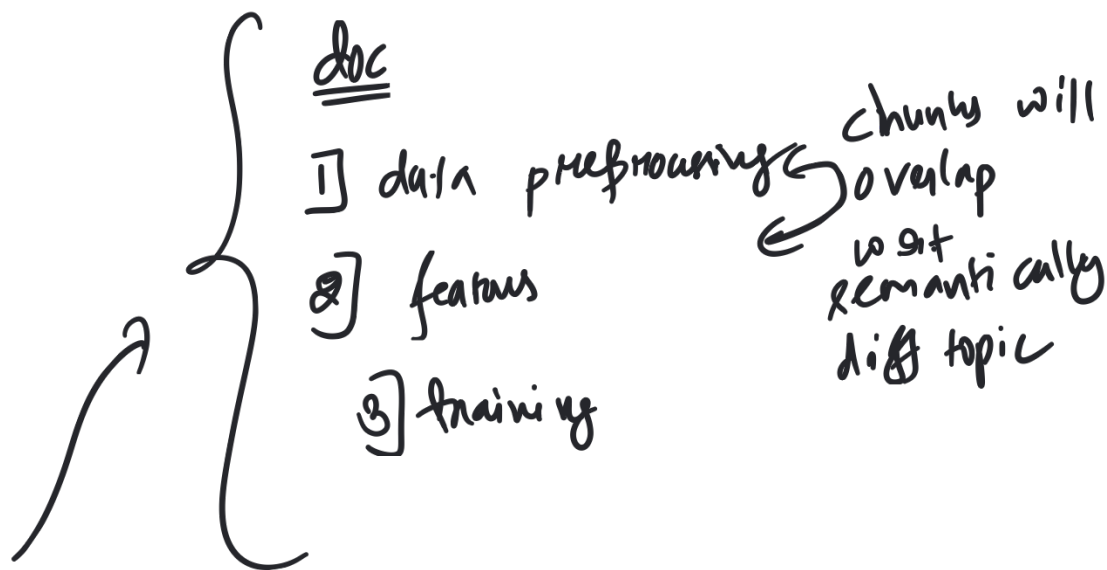
Poor RAG performance

- Chunks are large
(mixes multiple topics)
- Chunks are too small
(provides insufficient context)

Fixed size chunking

→ simple and fast
→ easy to implement

→ can break semantic



Semantic chunking
split it into conceptual
section

Chunk with semantic meanings intact in the chunk

Text (on sandwich)

Making a sandwich is easy.

First, take two slices of bread and either butter or mayonnaise. Next, apply your choice of spread on one side of each bread slice. Add your choice of fillings such as cheese, lettuce, tomato, and ham. Finally, put the bread slices together with the buttered sides facing inward. Cut the sandwich diagonally and serve it with chips and a pickle on the side.

Sandwiches are perfect for lunch or a quick snack. Popular sandwich varieties include BLT, grilled cheese, and club sandwich. They can be prepared in advance and wrapped in foil for later. Children often prefer peanut butter and jelly sandwiches.

[Chunk: Ingredients]

Making a sandwich is easy. First, take two slices of bread and either butter or mayonnaise. Add your choice of fillings such as cheese, lettuce, tomato, and ham.

[Chunk: Preparation]

Apply your choice of spread on one side of each bread slice. Add your choice of fillings. Finally, put the bread slices together with the buttered sides facing inward. Cut the sandwich diagonally and serve it with chips and a pickle on the side.

[Chunk: Serving]

Sandwiches are perfect for lunch or a quick snack. They can be prepared in advance and wrapped in foil for later.

[Chunk: Popularity]

Popular sandwich varieties include BLT, grilled cheese, and club sandwich. Children often prefer peanut butter and jelly sandwiches.

Best For: Chats, Essays, Notes

Optimized for meaning-based content grouping, regardless of document structure.

The Semantic Chunker analyzes the content to identify conceptually related segments, creating chunks based on topic similarity and semantic coherence.

Chunks - semantically related
issue - slow

Markdown

```
# RAH
## Intro to RAH
...
## Intro to VS
...
### FAISS
...
### Chroma
...
```

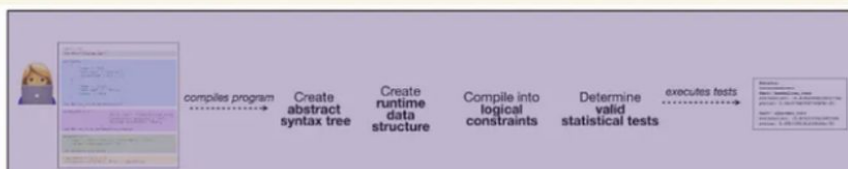


Figure 1: Overview of Tea's language constructs and its runtime system's compilation process. The colors correspond to the annotations provided in Figure 2.

ABSTRACT

Though statistical analyses are centered on research questions and hypotheses, current statistical analysis tools are not. Users must first translate their hypotheses into specific statistical tests and then perform API calls with functions and parameters. To do so accurately requires that users have statistical expertise. To lower this barrier to valid, replicable statistical analysis, we introduce Tea¹, a high-level declarative language and runtime system. In Tea, users express their study design, any parametric assumptions, and their hypotheses. Tea compiles these high-level specifications into a constraint satisfaction problem that determines the set of valid statistical tests, and then executes them to test the hypothesis. We evaluate Tea using a suite of statistical analyses drawn from popular tutorials. We show that Tea generally matches the choices of experts while automatically switching to non-parametric tests when parametric assumptions are not met. We simulate the effect of mistakes made by non-expert users and show that Tea automatically avoids both false negatives and false positives that could be produced by the application of incorrect statistical tests.

¹named after Fisher's "Lady Tasting Tea" experiment [11]

arXiv, April 2019, on-line
2019. <https://doi.org/10000001.0000001>

KEYWORDS

statistical analysis, declarative programming language

1 INTRODUCTION

The enormous variety of modern quantitative methods leaves researchers with the nontrivial task of matching analysis and design to the research question. - Ronald Fisher [11]

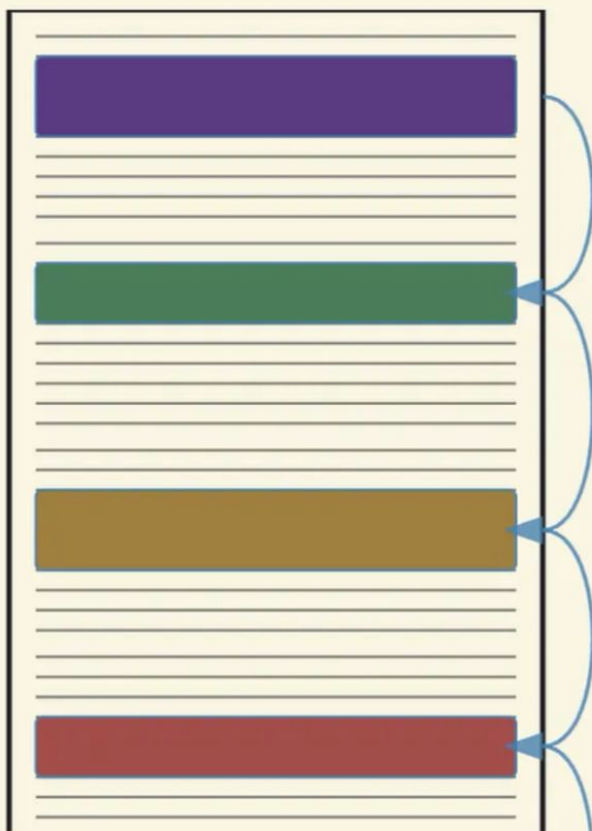
Since the development of modern statistical methods (e.g., Student's t-test, ANOVA, etc.), statisticians have acknowledged the difficulty of identifying which statistical tests people should use to answer their specific research questions. Almost a century later, choosing appropriate statistical tests for evaluating a hypothesis remains a challenge. As a consequence, errors in statistical analyses are common [20], especially given that data analysis has become a common task for people with little to no statistical expertise.

A wide variety of tools (such as SPSS [46], SAS [45], and JMP [43]), programming languages (R [44]), and libraries (including numpy [32], scipy [17], and statsmodels [37]), enable people to perform specific statistical tests, but they do not address the fundamental problem that users may not know which statistical test to perform and how to verify that specific assumptions about their data hold.

Best For: Structured Documents, PDFs, Markdown

The Recursive Chunker preserves the document's hierarchical structure, creating chunks that respect section boundaries and their nested relationships.

Ideal for documents where structure carries semantic meaning.

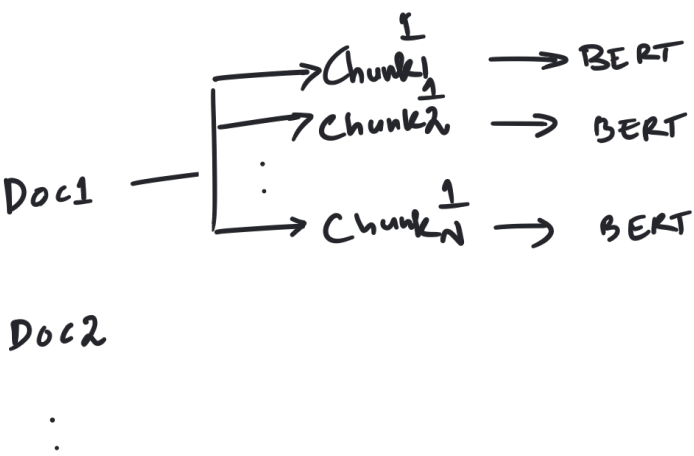


Best For: Books, Papers, Contracts, or other long documents

Late chunking embeds complete documents before segmentation, preserving their full semantic context for highly accurate retrieval.

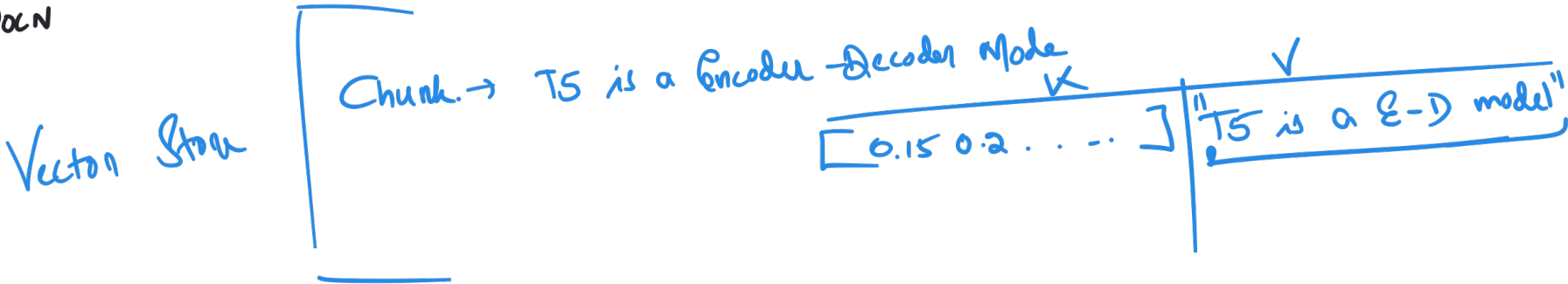
Capture cross-document relationships and complex topical connections that would otherwise be lost when documents are fragmented prematurely.

Vector DB

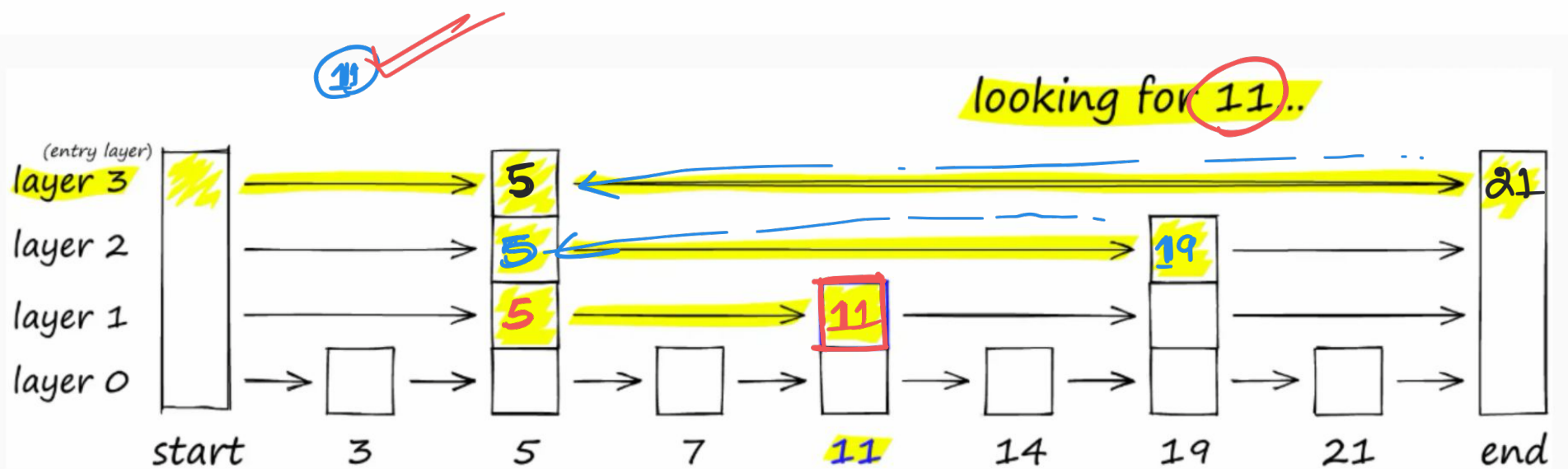


920

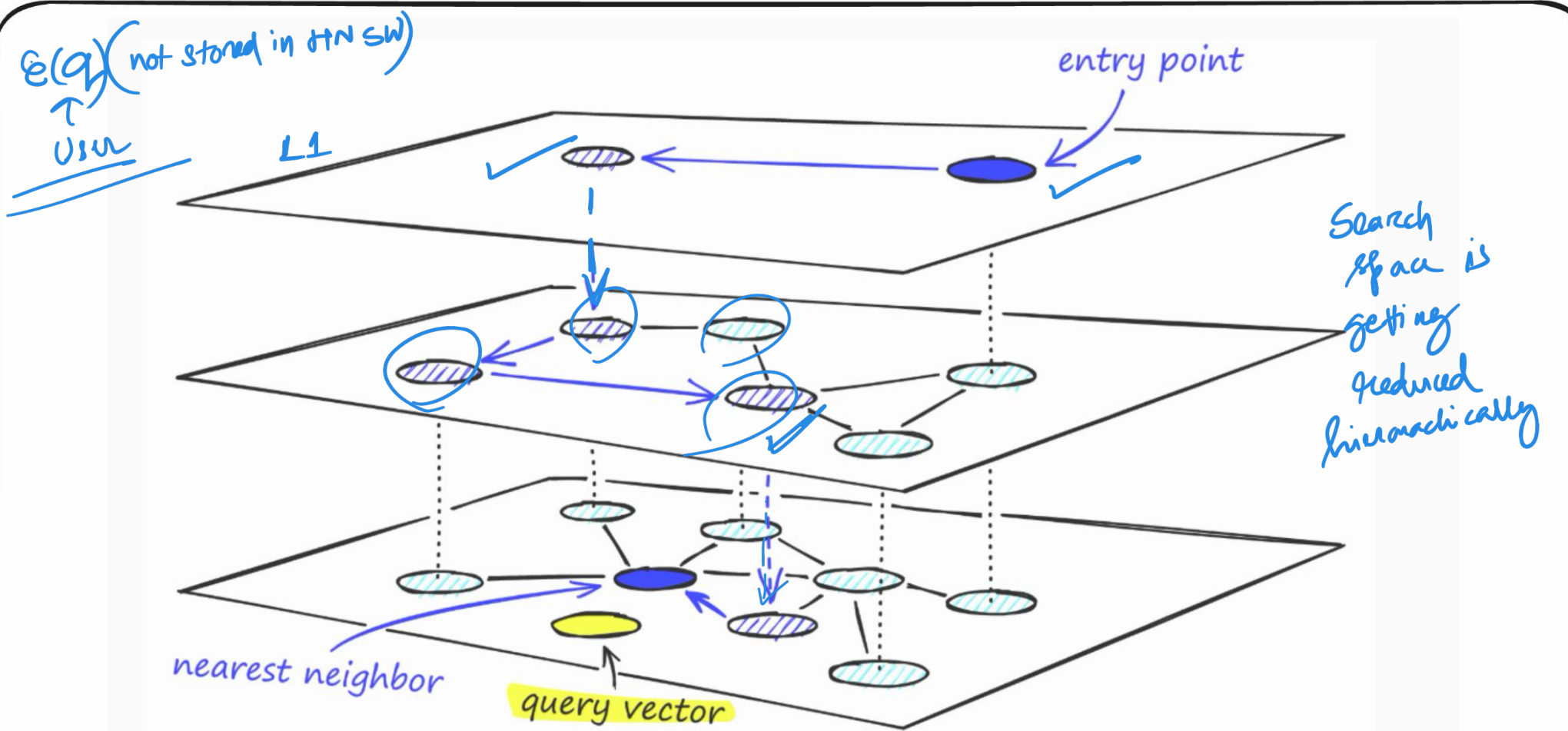
Key/Index	Value
$E(\text{Chunk}_1^{\text{Doc1}})$	Chunk ₁ ^{Doc1} ✓
$E(\text{Chunk}_2^{\text{Doc1}})$	Chunk ₂ ^{Doc1}
⋮	
$E(\text{Chunk}_N^{\text{DocN}})$	Chunk _N ^{DocN}



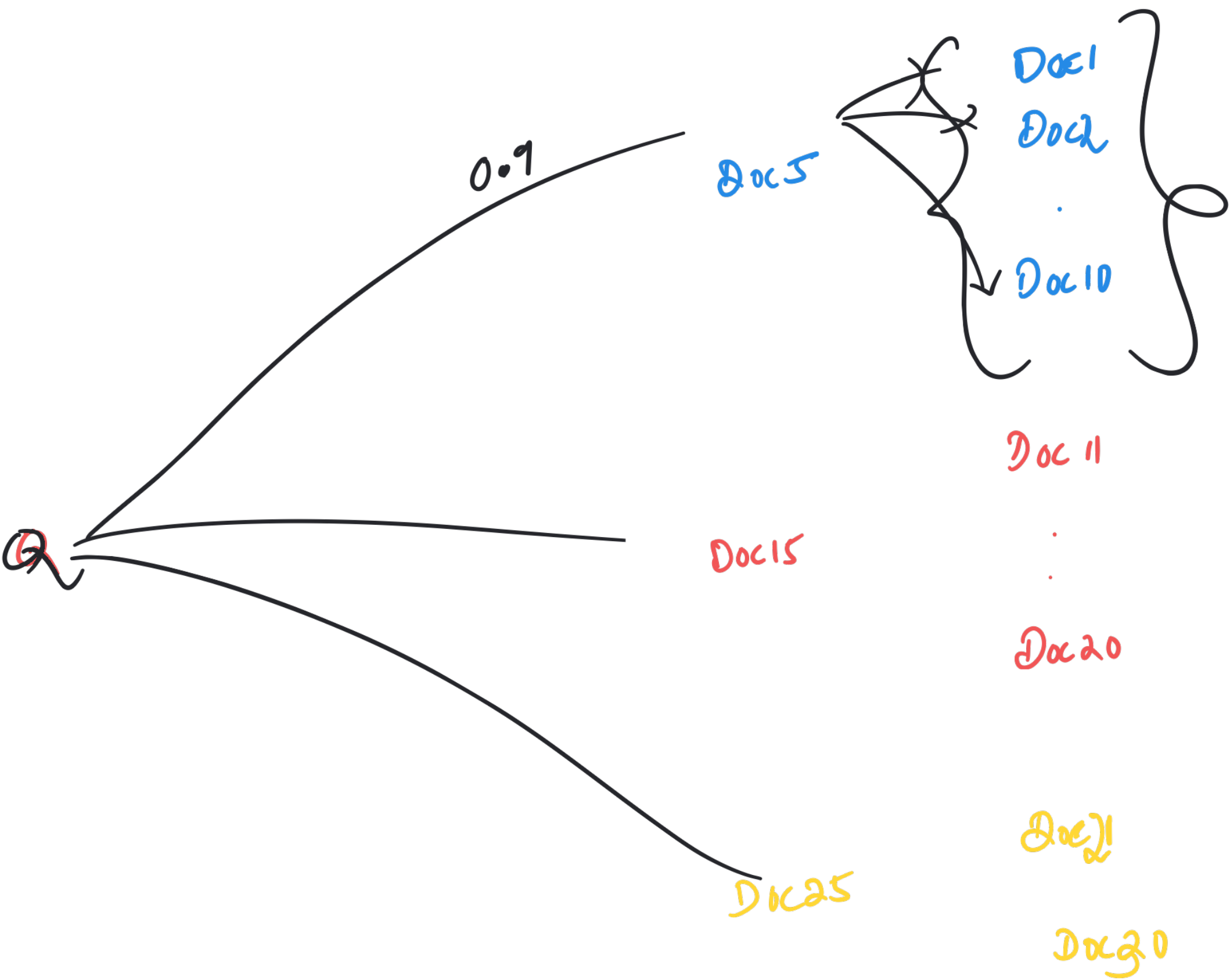
Intro HNSW $\Rightarrow$ combine with search ($O(\log N)$ time)
No. of chunks



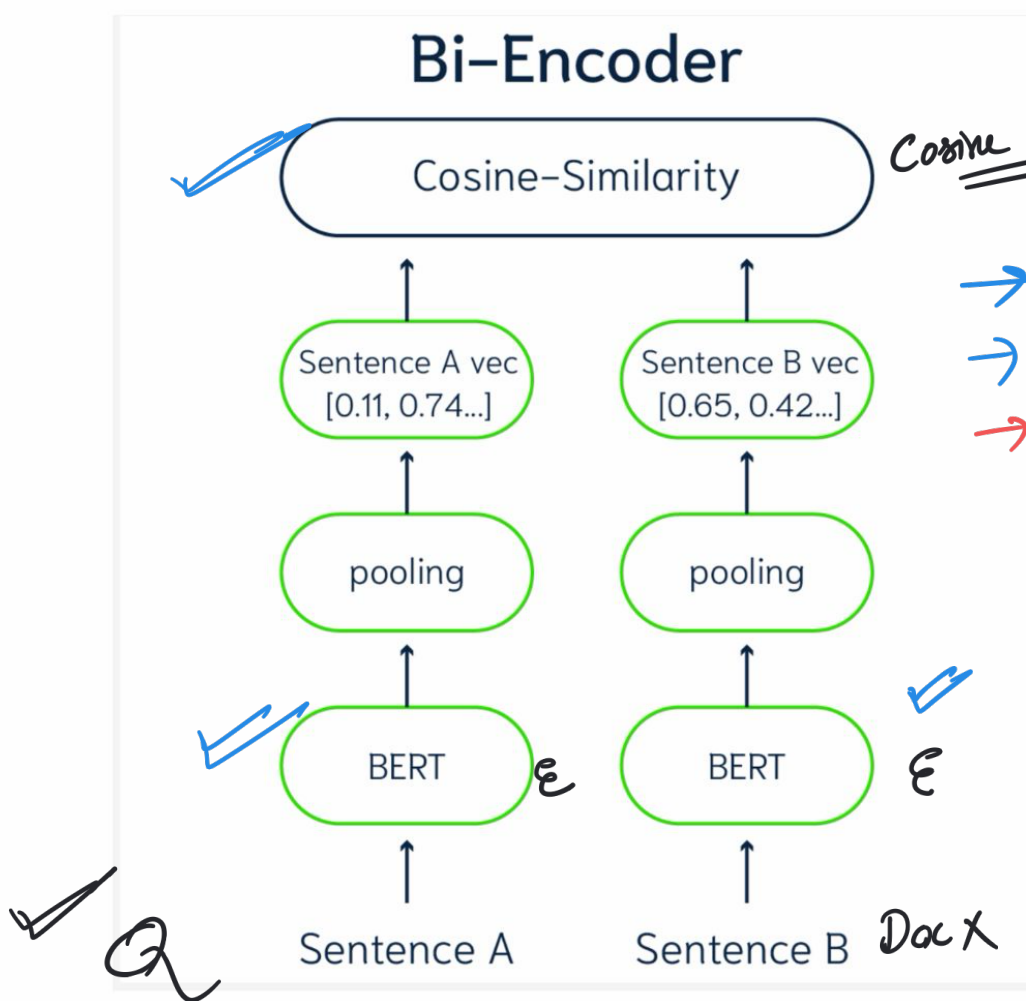
A probability skip list structure, we start on the top layer. If our current key is greater than the key we are searching for (or we reach end), we drop to the next layer.



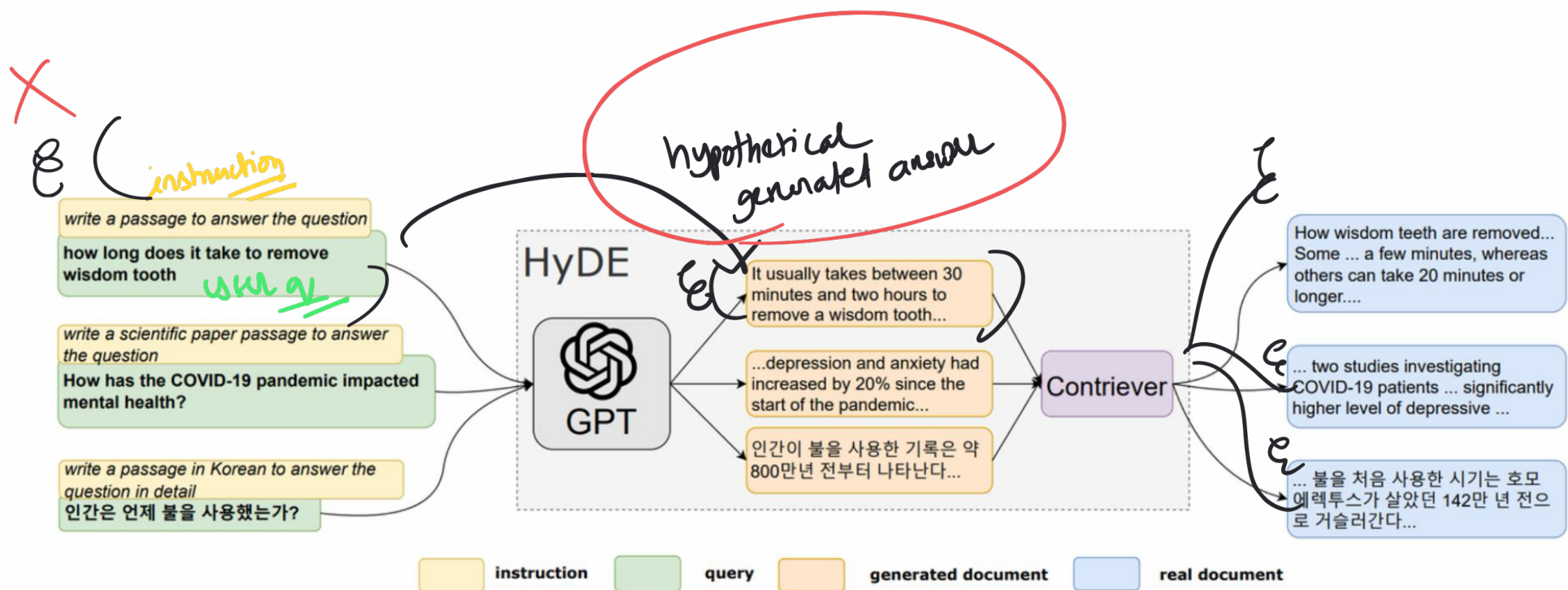
The search process through the multi-layer structure of an HNSW graph.



Siamese



→ Recall oriented
→ very fast
→ not very accurate



Cross-Encoder

95%
90%

0...1

Classifier

BERT

✓ Sentence A

✓ Sentence B

Q

Doc

→ extremely accurate
→ precise
→ slow

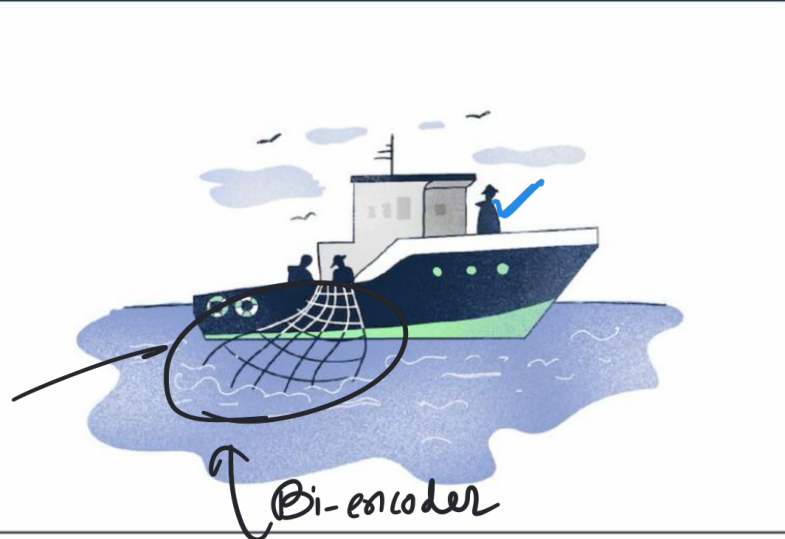
cross attention

only fuse

Step 1

→ million
→ 25/30

Query



Bi-encoder

Bi-Encoder

Fetch all the possible docs related to it

"top-k" = 25/30

Step 2

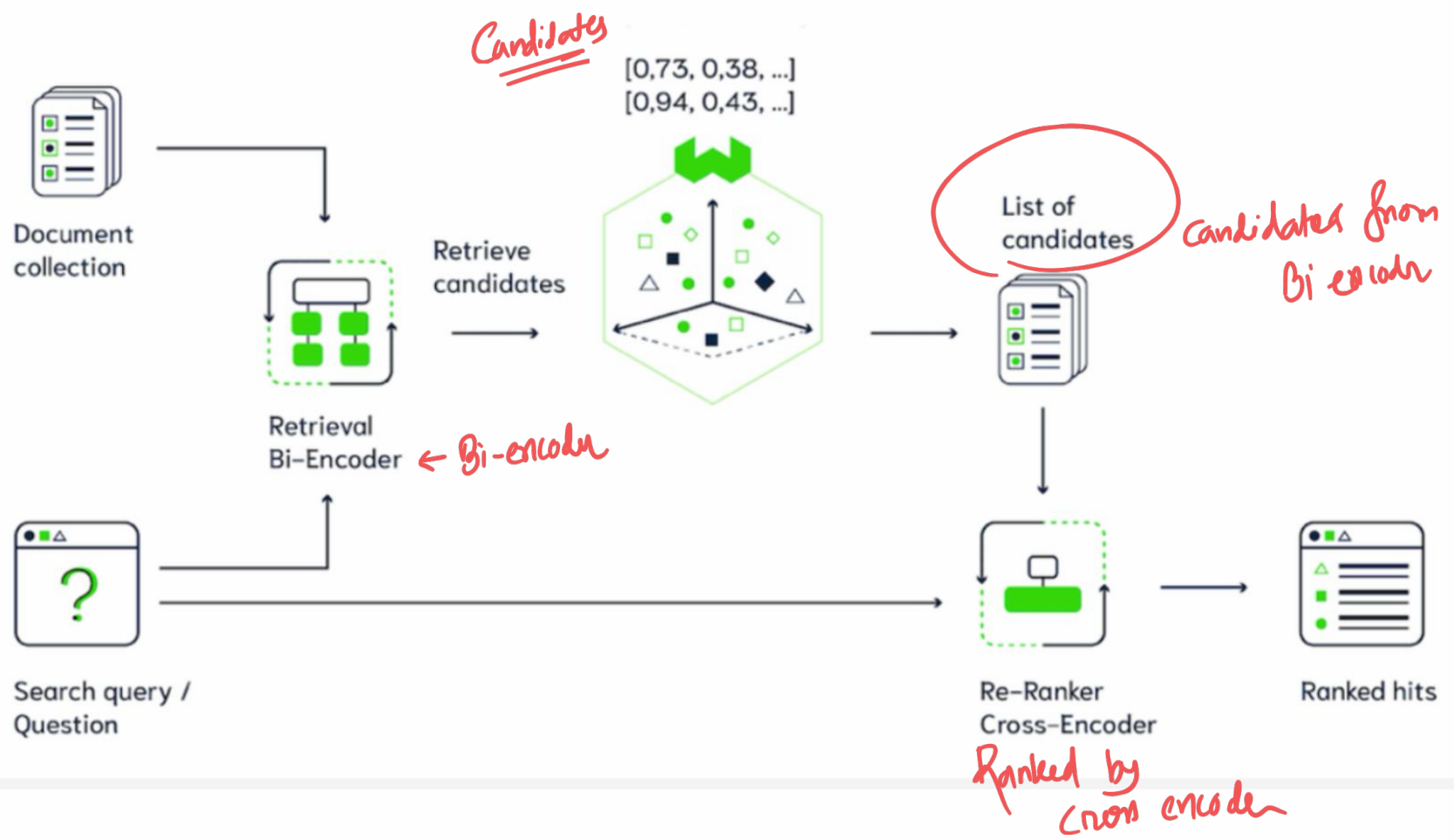


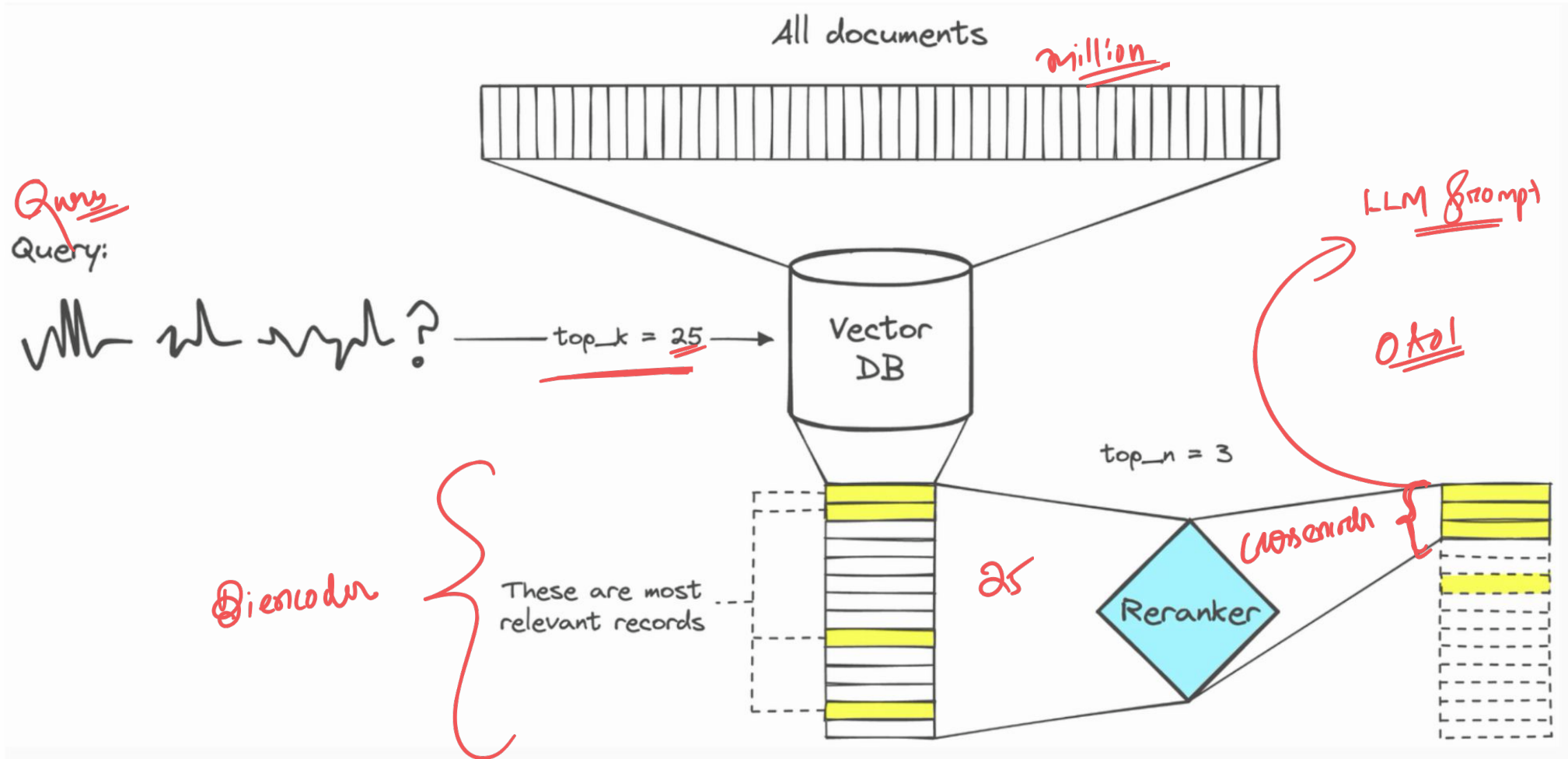
Bi-encoder of



Cross Encoder Reranker

"top-k" = 3/4





✓ Precision

maximize - precision

minimize - FPS

✓
Rank very accurately

slow

top-k Reranker (3/4)

find all possibly relevant items

✓ Recall

Bi-encoder - recall

minimize false negatives

Q - Train LLM

✓
Charles

Train LLM

...

fast

top-k (Bi-encoder) - 30/50

Quis short abstract

Hypothetical Document Embeddings

